DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

Iceberg learns to tidy up after streaming, and Grab squeezes a billion daily counts into one record each

Issue #7 | Sun, 11 Oct 2026 | ⏱️ ~42 min read | Level 3/5

🗄️ Today's two stories

1. **Apache Iceberg 1.12.0 shipped (tagged 30 Sep, announced 1 Oct, release post 5 Oct).** The headline for anyone streaming data into a lakehouse is a new Flink maintenance task that turns cheap-to-write but expensive-to-read "equality deletes" into compact deletion vectors. The release also spreads the v3 Variant and geospatial types to more engines, lays the first groundwork for format v4, and removes Spark 3.4 and Flink 2.0. We explain how Iceberg handles deletes on immutable files, why streaming writers create a mess for readers, how the clean-up works, and what to check before you upgrade.
2. **Grab moved the counting service behind its anti-fraud platform to Aerospike and halved its p99 read latency (InfoQ, 7 Oct).** About a billion requests a day used to cost four network round trips per event. The real win wasn't the database swap but a data-model change: thousands of tiny rows collapsed into one map per counter. We walk through why per-record index overhead decides schema design in Aerospike, how Grab migrated with shadow traffic and zero downtime, and the experiment that went wrong.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 Iceberg 1.12 cleans up after streaming writers

APACHE ICEBERG 1.12.0 · TAGGED 30 SEP 2026 · RELEASE POST 5 OCT · ~20 MIN READ

Here's an awkward truth about the modern lakehouse: the files at the bottom of it can never be changed. Every "update" or "delete" you run against an Iceberg table is really a note stuck on the side of an immutable file saying "ignore row 4,812" or "ignore anything where id = 42". Streaming writers love the second kind of note because it's cheap to write. Readers hate it because every query has to check it against every row. Iceberg 1.12.0 finally ships a built-in janitor that rewrites the expensive notes into cheap ones while your pipeline keeps running.

WHAT HAPPENED

The Apache Iceberg project released **Iceberg 1.12.0**. The GitHub release tag `apache-iceberg-1.12.0` is dated 30 September, and the release was announced on the dev mailing list on 1 October after the second release candidate passed its vote with 4 binding and 10 non-binding approvals.[1,2] The project's own release post followed on **5 October 2026**. It counts **777 commits from 142 contributors** since 1.11.0.[3]

The release post groups the changes into a handful of themes:[3]

- **Streaming deletes:** a new Flink maintenance task, `ConvertEqualityDeletes`, "resolves the equality deletes produced during streaming ingest into row-position deletion vectors". It needs table format version 3 or later and runs on Flink 1.20, 2.1, 2.2 and 2.3.
- **Variant and geospatial types:** the semi-structured `variant` type gets vectorized Parquet reads in Spark 4.0 and 4.1 and "shredded" writes from Flink, the Kafka Connect sink and the generic writer. The `geometry` and `geography` types can now be read and written in Parquet and Avro, but only Spark 4.1 has an end-to-end path.
- **Data layout:** Hilbert-curve clustering joins Z-order as a way to sort data files in Spark 4.1.
- **REST catalog:** new endpoints (list and load functions, unregister a table), remote-signing configuration, labels, and a spec for "read restrictions" that no reader enforces yet.
- **Format v4 groundwork:** manifest readers and writers for the v4 layout and relative paths in metadata. The post is blunt that "Iceberg 1.12.0 cannot read or write a V4 table".
- **Breaking changes:** Spark 3.4 and Flink 2.0 support are removed, a batch of deprecated APIs is gone, Java writers no longer produce an old style of delete file, and the default AWS SDK HTTP client moves to Apache HttpClient 5.

It also fixes two HIGH-severity `jackson-databind` CVEs (CVE-2026-54512 and CVE-2026-54513).[3]

None of these is a single flashy feature. But together they mark the point where Iceberg's v3 format, ratified in 2025, starts to work properly for the messiest workload there is: a stream of upserts landing in a table that analysts are querying at the same time. To see why that's hard, we need to start with how Iceberg deletes anything at all.

THE BACKGROUND YOU NEED

A table made of files that never change

Apache Iceberg *≈ an open table format: a specification for describing a database-style table as a set of data files plus metadata files in object storage, so many engines (Spark, Flink, Trino, Snowflake, Redshift…) can read and write the same table* doesn't store data itself. Your rows live in ordinary **Parquet** *≈ a columnar file format: values of each column are stored together, compressed, with min/max statistics so readers can skip chunks they don't need* (or Avro or ORC) files in a bucket. Iceberg adds a tree of metadata on top: a table metadata file points to **snapshots** *≈ immutable versions of the table; every commit creates a new one, which is how Iceberg offers time travel and isolation*, each snapshot points to a manifest list, and each **manifest** *≈ a metadata file listing a batch of data or delete files along with per-column statistics, so a query planner can skip files without opening them* lists the files that make up the table at that moment.

The data files are never edited in place. That's a deliberate design choice: object stores like S3 don't support in-place edits anyway, and immutable files make caching, concurrent readers and time travel simple.

The spec describes the format's evolution in one line per version. Version 1 "defines how to manage large analytic tables using immutable file formats". Version 2 "adds row-level updates and deletes for analytic tables with immutable files". Version 3 "extends data types and existing metadata structures to add new capabilities". Version 4 "restructures metadata", but it "is under active development and has not been formally adopted".[5]

Deleting from an immutable file: two kinds of notes

So how do you delete one row from a 500 MB Parquet file you're not allowed to touch? You have two options.

Copy-on-write: rewrite the whole file without that row. Readers stay fast, but a one-row delete costs a 500 MB rewrite.

Merge-on-read: leave the data file alone and write a small **delete file** *≈ a separate file that records which rows of the table's data files should be treated as deleted; readers merge it with the data at query time* next to it. The write is cheap; the reader pays later by applying the deletes as it scans.

Format v2 introduced two flavours of delete file for merge-on-read, and the spec defines them in one sentence each:[5]

- **Position deletes** "mark a row deleted by data file path and the row position in the data file". In other words: "in file `a.parquet`, row 4,812 is gone".
- **Equality deletes** "mark a row deleted by one or more column values, like id = 5". In other words: "any row, in any older file, whose `id` is 5 is gone".

These look similar, but they put the cost in completely different places.

To write a position delete, the writer first has to **find** the row: which file is it in, and at what position? For a batch job that just scanned the table, that's free. For a streaming job receiving "customer 42 changed their address" from a **CDC** *≈ change data capture: streaming every insert, update and delete from a source database's log so another system can mirror it* feed, it's expensive, because the job would need to look up where customer 42 currently lives among thousands of files.

An equality delete skips that lookup entirely. The writer simply records "id = 42 is deleted" and appends the new version of the row to a fresh data file. That's why streaming engines like **Apache Flink** *≈ a distributed stream-processing engine that runs continuous jobs over unbounded data, often used to write CDC streams into lakehouse tables* use equality deletes for **upserts** *≈ an "update or insert" operation: if a row with this key exists, replace it; otherwise add it*.

The bill arrives at read time. A reader can't skip anything: it has to compare every row it scans against the set of deleted key values. Alex Merced of Dremio puts it well: equality deletes "convert reads into join-like work".[7] And the work accumulates. Every commit can add more delete files, so a table fed by a busy stream gets slower to read with every checkpoint, until someone runs **compaction** *≈ a maintenance job that rewrites many small data and delete files into fewer, larger, clean data files*.

Sequence numbers: which notes apply to which files?

There's one more subtlety. An equality delete for `id = 42` must remove the *old* row with id 42, but not the *new* row the same commit just wrote. Iceberg handles this with **data sequence numbers** *≈ a counter assigned to each file at commit time, increasing with every commit, used to decide which delete files apply to which data files*.

The spec's scan-planning rules are precise about it. An equality delete applies to a data file only when "the data file's data sequence number is *strictly less than* the delete's data sequence number". A position delete or deletion vector applies when the data file's number is "*less than or equal to*" the delete's, because it names exact positions and so can safely target rows written in the same commit.[6] Equality deletes written with an

unpartitioned spec "are applied as global deletes", meaning to every older data file in the table.[6] That last rule is what makes them so expensive: a single equality delete file can force a check against the whole table.

What v3 changed: deletion vectors

Position delete files had their own problem. Each commit could add another one, so a single data file could end up with dozens of small delete files pointing at it, all of which a reader had to open and merge.

Format v3, whose ratification Snowflake's engineering blog celebrated in June 2025, replaced them with **deletion vectors** *≈ a compact bitmap, one per data file, in which bit N set means "row N of that file is deleted"*.[8] They're stored as **Roaring bitmaps** *≈ a compressed bitmap format that splits the number space into chunks and picks the most compact encoding (array, bitmap or run-length) for each chunk* inside **Puffin** *≈ Iceberg's sidecar file format for blobs of auxiliary data such as statistics sketches and deletion vectors* files.[8,9] Several vectors can share one Puffin file, and a data file has at most one vector per snapshot, so a writer adding deletes must merge them into the existing bitmap.[7] The spec reflects this: a newly added vector "must contain all deletes from existing position delete files" for that data file, and manifests must record its exact offset and length within the Puffin file.[6]

The effect is that the read cost per data file stays bounded: one small bitmap, no matter how many commits have deleted rows from it. Iceberg 1.10.0 (September 2025) brought deletion vectors, row lineage, Variant and geospatial types to the Java library.[9] Iceberg 1.11.0 followed in May 2026, adding, among other things, Flink support for v3's nanosecond, variant and unknown types.[12] Since then the big engines have followed: Snowflake made v3 generally available on 7 May 2026, including deletion vectors, Variant and geospatial types (though external engines can't yet write to Snowflake-managed v3 tables),[10] and Amazon Redshift added v3 support, with deletion vectors, row lineage and default values, on 31 August 2026.[11]

But notice what v3 didn't fix. Deletion vectors replaced *position* deletes. *Equality* deletes are still there in v3, because streaming writers still can't afford to look up row positions. So a Flink upsert pipeline writing to a shiny v3 table was still producing the expensive kind of note.

How Iceberg learned to delete



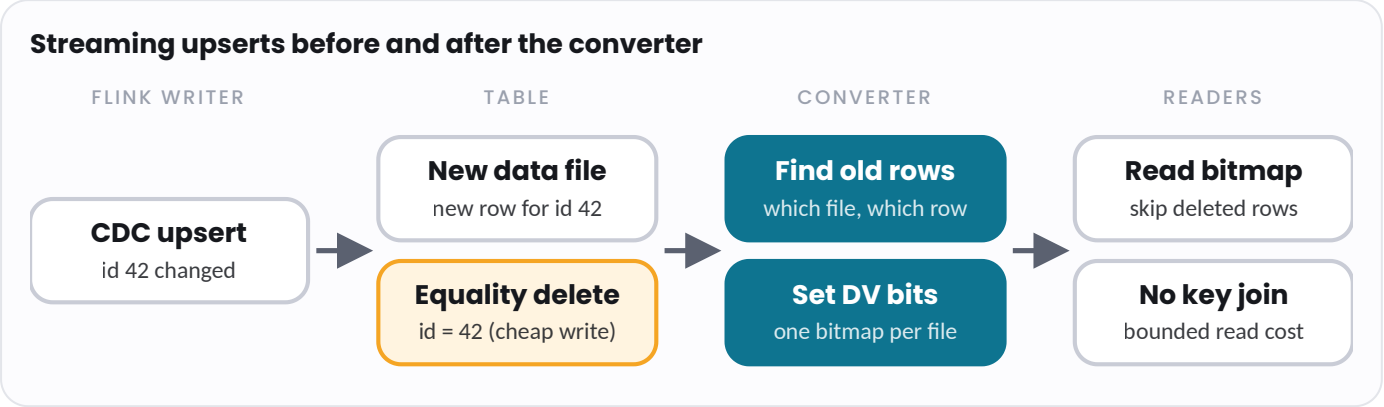
From the Iceberg spec, Snowflake (Jun 2025), Dremio on 1.10.0, Google on 1.11.0 and the 1.12.0 release.

HOW IT WORKS

The janitor: converting equality deletes into deletion vectors

`ConvertEqualityDeletes` is a task for Iceberg's Flink **TableMaintenance** *≈ Iceberg's framework for running table upkeep (expiring snapshots, compacting files, removing orphans) as part of a long-running Flink job instead of as separate batch jobs* framework. In the release post's words, it "resolves the equality deletes produced during streaming ingest into row-position deletion vectors". It pairs with Flink's `IcebergSink`, and "the converter resolves those into deletion vectors and commits to the target branch, or converts in place".[3] The GitHub changelog shows it arriving as a set of pieces: conversion operators, a planner, a committer and the integration itself.[1]

Conceptually, "resolving" an equality delete means doing the lookup the streaming writer skipped. For a delete that says `id = 42`, the converter has to find which older data files contain a row with `id = 42` and at which positions, then set those bits in each file's deletion vector. Once that's committed, the equality delete file is no longer needed, and every subsequent reader gets the cheap, bounded bitmap instead of the join-like key comparison. (This description of the mechanics is our explanation of what "resolving into row-position deletion vectors" has to involve; the release post doesn't spell out the algorithm.)



This is the classic trade-off in log-structured systems, the same one databases make with **LSM trees** *≈ log-structured merge trees: storage engines (RocksDB, Cassandra) that make writes cheap by appending, then pay with background compaction to keep reads fast*: make the hot path cheap by deferring work, then do that work in the background before it hurts anyone. Before 1.12, Iceberg users had to build that background step themselves, usually with a periodic Spark compaction job that rewrote whole data files. The new task does a lighter job, rewriting deletes rather than data, inside the same Flink deployment that's doing the writing.

The branch option matters more than it looks. Iceberg tables support **branches** *≈ named, independent lines of snapshots within one table, like Git branches, so a writer can commit to a branch and publish it to main later*. Converting on a branch lets you resolve deletes away from `main` and publish the clean result, rather than having the converter's commits race with the streaming writer's commits on the same branch.[3]

🔑 Equality delete

- Says: delete any older row where id = 42
- Writer needs no lookup, so it's cheap
- Reader compares every scanned row with the keys
- Unpartitioned ones apply to the whole table
- Applies only to strictly older files

VS

🧮 Deletion vector (v3)

- Says: in file a.parquet, rows 7 and 4,812 are gone
- Writer must know positions and merge bitmaps
- Reader checks one bitmap per data file
- At most one per data file per snapshot
- Stored as Roaring bitmaps in Puffin files

Variant: JSON that the engine can see inside

The second big theme is the **Variant** *≈ a v3 column type for semi-structured, JSON-like data whose shape can vary from row to row* type. The spec describes it as a value that "stores semi-structured data", "similar to JSON with a wider set of primitive values".[5]

Why not just store JSON as a string? Because to read one field from a JSON string, an engine has to parse the whole thing, row after row. Variant, which Parquet added as a logical type in August 2025, stores two binary fields instead: a `metadata` field holding a dictionary of field names, so a name repeated in every row is stored once, and a `value` field holding the data in a compact binary encoding that lets engines "navigate nested structures and extract only the fields they need". Types survive too: "timestamp values are stored as integers rather than being stored as strings".[13]

Shredding *≈ pulling frequently used fields out of a Variant into their own typed Parquet columns, while keeping the rest as binary* goes a step further. If most rows have an `event_ts` field that's a timestamp, the writer can store it as a real `int64` timestamp column inside the Variant's Parquet group. Parquet's shredding spec gives exactly this example:[14]

A Variant column in Parquet, unshredded vs shredded

Unshredded: one opaque blob

```
optional group event (VARIANT) {
  required binary metadata;
  required binary value;
}
# every query on event_ts
# decodes the whole value
```

Shredded: typed sub-columns

```
optional group event (VARIANT) {
  required binary metadata;
  optional binary value;
  optional group typed_value {
    required group event_ts {
      optional binary value;
      optional int64 typed_value
        (TIMESTAMP(true, MICROS));
    }
  }
}
```

Right pane simplified from the Parquet Variant Shredding spec; the spec's example also shreds event_type.

The payoff, per the spec: a query like `variant_get(event, '$.event_ts', 'timestamp')` reads only the `event_ts` column; statistics on a shredded column can skip files, row groups and pages; and a filter on a shredded field can run first, so the rest of the Variant is loaded only for matching rows.[14]

Iceberg 1.12 adds one important rule: "a field is shredded into a typed column only when all of its values fall into a single type family".[3] That sounds fussy, but it protects the statistics. Parquet's spec notes that data skipping only works "when `value` is always null", meaning every value actually landed in the typed column,[14] and the Iceberg spec says bounds "must not be written to describe values with mixed Variant types (other than null)".[6] Shredding a field whose values are sometimes strings and sometimes numbers would give you a typed column full of holes and no usable min/max statistics.

Geospatial, Hilbert curves and v4 groundwork

The `geometry` and `geography` types (stored as **WKB** *≈ Well-Known Binary, the standard binary encoding for points, lines and polygons*) can now be written and read in Parquet and Avro. But the limits are real: "Support is limited to Spark 4.1 (not Spark 3.5 or 4.0, Flink, or ORC)", reads use the row-based reader, and "there are no spatial predicates yet, so filters are expressed against non-geospatial columns".[3] In other words, you can store and round-trip spatial data, but Iceberg won't yet skip files based on a bounding box in your `WHERE` clause.

For data layout, Spark's `rewrite_data_files` procedure can now cluster by a **Hilbert curve** *≈ a space-filling curve that maps multi-dimensional values to one dimension while keeping nearby points close together, so data sorted along it clusters well on several columns at once*, alongside the existing **Z-order** *≈ another space-filling curve used to sort data by several columns at once; simpler than Hilbert but with more "jumps" between distant regions*. The release post's example:[3]

```
CALL system.rewrite_data_files(
  table => 'db.tbl',
  strategy => 'sort',
  sort_order => 'hilbert(c1, c2)'
);
```

Why bother? If your queries filter on two columns, say `lat` and `lon`, or `tenant_id` and `event_date`, a plain sort on the first column leaves the second one scattered. A space-filling curve keeps rows that are close on *both* columns physically close, so the per-file min/max statistics become tight on both, and more files can be skipped. Hilbert curves generally preserve locality better than Z-order; that's the reason to offer both. It ships for Spark 4.1 only.[3]

Finally, v4. The 1.12 release includes a v4 manifest reader and writers, an adapter so v4 files can surface through today's `ManifestFile` and `DataFile` interfaces, **relative paths in metadata** (a step towards tables you can move between buckets without rewriting metadata), a content-statistics addition to the spec, and a read-only implementation of the "Mumbling bitmap", a bounded-size compressed bitmap based on Roaring. But again: 1.12.0 "cannot read or write a V4 table".[3] Merced notes that v4 discussions aim to retire equality deletes entirely.[7] The converter in 1.12 is the practical bridge until then.

WHAT IT MEANS FOR YOU

If you run Flink upsert or CDC pipelines into Iceberg, this is the release to evaluate. The pattern the community has been recommending, using equality deletes only as a short-lived landing format and resolving them quickly,[7] now has a built-in implementation. The catch is that it needs **format v3** tables. Before you move a table to v3, remember that, as the Snowflake and Redshift notes show, not every engine that reads your table supports every v3 feature in the same way yet.[10,11] Check every reader before you flip `format-version`.

If you run Spark 3.4 or Flink 2.0, 1.12 is not for you yet. There's no 1.12 build of `iceberg-spark-runtime-3.4`; supported lines are Spark 3.5, 4.0 and 4.1, and Flink 1.20, 2.1, 2.2 and 2.3.[3,4]

If you use Kafka Connect's Iceberg sink, the upgrade fixes something nasty: the sink now "surfaces commit failures instead of swallowing them", "adds bounded retry for transient commit exceptions" and adds "a metric for partial commit failures".[3] If you've ever had a connector that looked healthy while silently not committing, this is for you. Dremio also flags that custom transforms using removed members of `TableReference` and `IcebergWriterResult` may stop compiling.[4]

If you manage the AWS SDK on your own classpath, you "must switch from `software.amazon.awssdk:apache-client` to `software.amazon.awssdk:apache5-client`".[3]

Dremio's upgrade guide has a sensible rule of thumb: "upgrade by table behavior, not by dependency file". Keep three decisions separate: the Java library version, the engine runtime, and the table format version. It suggests upgrading stateless readers first, then low-volume writers, then streaming writers, and maintenance jobs last.[4]

> 💥 **Watch out for**
>
> - **Old-style position deletes are now read-only.** Java writers no longer create "position delete files with row data" (delete files that also carry a copy of the deleted row). Existing ones stay readable, but `rewrite_position_delete_files` and `rewrite_table_path` reject them. Dremio suggests checking the `position_deletes` metadata table for non-null `row` values before upgrading.[4]
> - **Read restrictions are spec-only.** A REST catalog can now return column masks and row filters in `loadTable`, and the contract says a reader "must fail the query rather than return raw, partial, or empty rows". But today such a response "is silently ignored by every consumer".[3] Don't rely on it for access control yet.
> - **REST path encoding changed** to follow RFC 3986, which can affect proxies or custom catalogs that decode paths differently.[4]
> - **Don't enable v4.** It's groundwork only; `format-version=4` is not a supported path.[4]
> - **Geospatial is Spark 4.1-only** and has no spatial predicates yet.[3]

WHAT HAPPENS NEXT

Release planning for 1.13 has already started on the dev list, with an early-January target proposed and a volunteer already lined up to manage 1.14.[2] Work on v4 continues in the weekly syncs, including file and vector types and a proposal to make Vortex an Iceberg file format.[2] For streaming users, the thing to watch is how quickly managed platforms (Snowflake, Redshift, S3 Tables, Databricks' Iceberg support) pick up 1.12's converter or build their own equivalent, and whether v4 really does retire equality deletes.

📖 Words from this story

Apache Iceberg: an open table format that describes a table as data files plus metadata files in object storage, so many engines can share it.

Parquet: a columnar file format with per-chunk statistics that let readers skip data.

Snapshot: an immutable version of an Iceberg table; every commit creates one.

Manifest: a metadata file listing data or delete files with per-column statistics.

Delete file: a separate file recording which rows to treat as deleted, merged in at read time.

CDC (change data capture): streaming every insert, update and delete from a database's log to another system.

Apache Flink: a distributed engine for continuous stream-processing jobs.

Upsert: update the row with this key if it exists, insert it otherwise.

Compaction: rewriting many small files into fewer, clean ones.

Data sequence number: a per-commit counter that decides which deletes apply to which data files.

Deletion vector: a per-data-file bitmap marking deleted row positions (v3).

Roaring bitmap: a compressed bitmap that picks the best encoding for each chunk of the number space.

Puffin: Iceberg's sidecar file format for blobs like statistics and deletion vectors.

TableMaintenance: Iceberg's framework for running table upkeep inside a Flink job.

LSM tree: a storage engine design that appends writes and compacts in the background.

Branch: a named, independent line of snapshots within one table.

Variant: a v3 column type for semi-structured, JSON-like data.

Shredding: storing common Variant fields as their own typed Parquet columns.

WKB (Well-Known Binary): the standard binary encoding for geometries.

Hilbert curve: a space-filling curve that keeps multi-column neighbours close when sorted.

Z-order: a simpler space-filling curve for multi-column sorting.

STORY 2 Grab's billion-a-day counter, rebuilt around one map per key

ARCHITECTURE CASE STUDY · GRAB COUNTER SERVICE → AEROSPIKE · GRAB BLOG 3 JUL 2026, INFOQ 7 OCT · ~16 MIN READ

How many times has this card failed a payment in the last fifteen minutes? How many rides has this account requested today? Questions like these sit in the path of almost every transaction at Grab, Southeast Asia's ride-hailing, delivery and payments company, because the answers feed its fraud checks. The service that answers them handles about a billion requests a day. Grab just halved its worst-case latency, and the most interesting part is that the new database wasn't what did it.

WHAT HAPPENED

Grab moved the storage behind its **Counter Service** from a wide-column database to **Aerospike**, redesigned the data model on the way, and migrated live traffic with no downtime. The engineering team (Jia Long Loh, Md Riyadh, Runze Cui and Yuanzhe Liu) described the project on Grab's tech blog on **3 July 2026**,[15] and InfoQ brought it to a wider audience on **7 October 2026**.[16]

Counter Service answers time-windowed count queries for Grab's anti-fraud platform, such as recent ride requests by a user or failed payment attempts on a card. It handles "tens of thousands" of queries per second, about a billion requests per day, with strict latency and reliability requirements.[15]

Grab reports these results:[15,16]

- Production **p99 read latency down by roughly 50%** across read paths.
- **On-disk data down from about 3 TB to about 1 TB.**
- **Cost per node down by roughly 45–50%.**
- Every counter update became **one atomic operation** instead of a read-modify-write.
- **Replication factor reduced from 3 to 2**, saving about a third of storage and index memory.
- Zero downtime and no data-integrity issues during the migration.

These are Grab's own numbers; InfoQ attributes them to Grab and didn't verify them independently.[16] The team's own conclusion is the most useful line in the post: most of the gains came from the schema redesign, collapsing rows into maps, not from the database swap alone.[15]

THE BACKGROUND YOU NEED

What a counter service does, and why it's hard

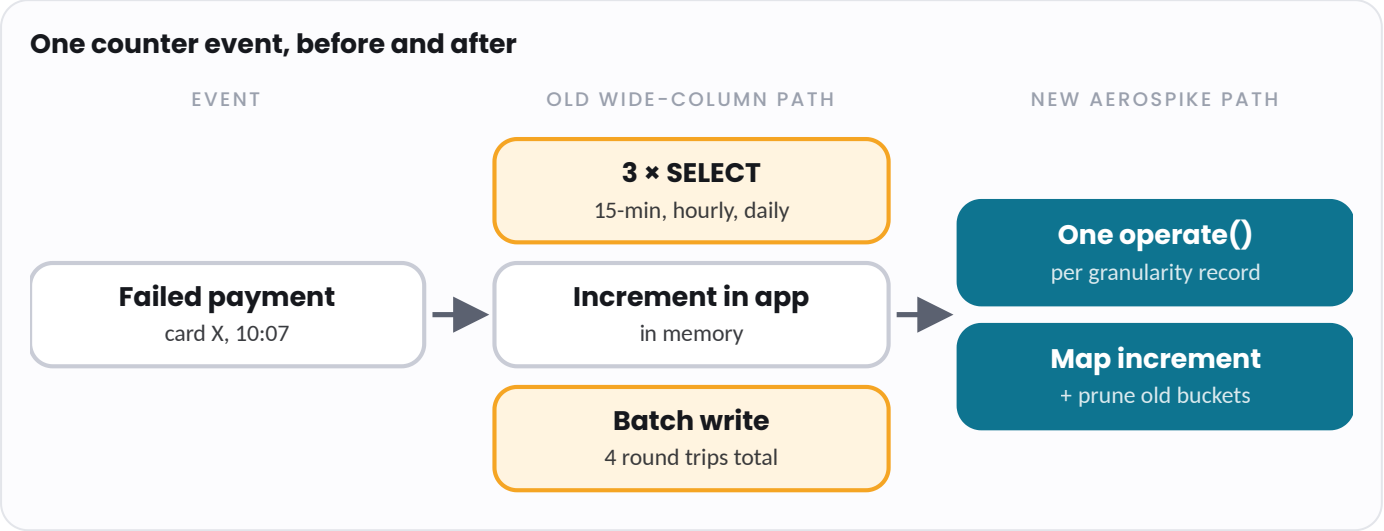
A fraud rule might say "block if this card has more than five failed payments in the last hour". To evaluate it, you need a fast, always-up-to-date answer to "how many failed payments in the last hour?". You can't scan an event log at query time with a billion requests a day, so you keep pre-aggregated counts in time buckets: one count per 15 minutes, one per hour, one per day. A one-hour window becomes "add up four 15-minute buckets", and a seven-day window becomes "add up seven daily buckets".

Every new event has to increment the right buckets, and every query has to sum the right ones, quickly, while both happen tens of thousands of times a second.

The old design: rows sorted by time

Grab doesn't name the old database. It describes it as a **wide-column database**, with data in three tables (15-minute, hourly and daily), each keyed by a **partition key** *≈ the part of the primary key that decides which node stores a row; all rows with the same partition key live together* plus a timestamp **clustering column** *≈ a primary-key column that sorts rows within a partition on disk, so a range of them (say, a time window) can be read in one go*.[15] That's the classic Cassandra-style layout: Cassandra's docs describe the partition key as the column by which "the data is divided and stored", and `CLUSTERING ORDER BY` as a way to "order rows storage to make use of the on-disk sorting of columns".[17]

It's a perfectly reasonable design for time series. Reading "the last hour for card X" is a range scan within one partition. The problem was the write path. Each counter event triggered a **read-modify-write** *≈ reading a value, changing it in application memory, then writing it back; three steps where other writers can interleave*: three parallel `SELECT`s (one per granularity), an increment in memory, then a batch write. That's **four network round trips per event**.[15]



Read-modify-write has a second cost besides latency. Between the read and the write, another event for the same card can do its own read, and one of the two increments can be lost unless the database or the application adds some form of locking or conditional write. For a fraud counter, an undercount is exactly the wrong failure.

Aerospike in five minutes

Aerospike *≈ a distributed key-value database designed for predictable low latency at high throughput, typically keeping its index in RAM and its data on SSDs* is built around a few ideas that matter for this story.

Records, bins and namespaces. Data is stored as **records** *≈ Aerospike's unit of storage: a key plus a set of named fields, roughly a row*, each identified by a key and made up of **bins** *≈ Aerospike's name for the named fields inside a record*. A

bin can hold a scalar or a **collection data type** ≈ *a list or map stored inside a single bin, which the server can modify in place with dedicated operations* such as a map.

The primary index lives in RAM. In the default **Hybrid Memory Architecture** ≈ *Aerospike's default layout: the primary index in memory, the record data on SSD*, "the primary index is stored in memory, and data is stored on the SSD".[18] Looking up a key is an in-memory tree lookup followed by a read of the record from flash. Under the hood, keys are hashed into "a total of 4096 partitions" spread across the cluster, and each partition's index is a set of in-memory **red-black trees** ≈ *self-balancing binary search trees that keep lookups, inserts and deletes logarithmic* Aerospike calls *sprigs*.[19] Enterprise Edition can also put the index on persistent memory or an NVMe SSD; Community Edition keeps it in process memory.[19] Remember that option; Grab tried it.

Every record costs index memory, no matter how small it is. This is the fact the whole redesign turns on. Each record carries a fixed cost of "64 bytes of metadata", including "the 20-byte digest, generation counter, void-time (expiration), last-update time", and the total is "multiplied by the replication factor for the namespace".[20] A record holding a single 8-byte counter still costs 64 bytes of RAM per copy.

To make that concrete with an illustrative number (ours, not Grab's): one billion records at 64 bytes each is 64 GB of index, and at a **replication factor** ≈ *how many copies of each record the cluster keeps on different nodes* of 3 that's 192 GB of RAM across the cluster, before storing a single byte of actual data.

Many children? Use one record with a map. Aerospike's own data-modelling guide makes consolidation the default: "store one record per parent entity with a Map or List bin holding the children".[20] It caps record size, though: the namespace `max-record-size` defaults to 1 MiB and can go up to 8 MiB.[20]

HOW IT WORKS

Step 1: the data model, three attempts

Grab's first instinct was to keep the old shape: **one record per time bucket**. The team rejected it, along with a variant that also needed a **secondary index** ≈ *an extra index on a non-key field, letting you query by that field at the cost of more memory and write work*. The reason was the 64-byte index entry. With billions of records, index memory, not disk or CPU, became the constraint.[15]

The design they chose (their "approach 3") is **one record per counter per granularity**, holding a **sorted map** ≈ *a map whose entries are kept ordered by key, so lookups use binary search and key ranges can be addressed directly* from bucket timestamp (in milliseconds) to count.[15] Aerospike's docs describe a key-ordered map as one where "elements are maintained in key order. Key lookup uses binary search."[21]

Grab says this cut the number of records by more than 10x and reduced disk use, since the long counter key is now stored once per record rather than once per bucket.[15]

One card's hourly counts: rows vs one map (illustrative)

One record per bucket

```
card:X|h|09:00  -> 3
card:X|h|10:00  -> 1
card:X|h|11:00  -> 4
... one 64 B index entry
    per bucket, per replica
long key stored every time
```

One record, sorted map

```
key: card:X|hourly
bin counts (key-ordered map):
  { 09:00 -> 3,
    10:00 -> 1,
    11:00 -> 4, ... }
one 64 B index entry
key stored once
```

Key formats are invented for illustration. The record-per-counter-per-granularity design and the 64-byte index cost are from Grab and Aerospike's docs.

Step 2: one atomic write instead of four round trips

With a map per counter, an event becomes a single server-side operation on each affected record. Grab's write combines three things in one atomic call: a **map increment** on the current bucket, **pruning of stale entries by key range**, and a put of the counter key.[15]

Aerospike makes that safe because `operate` "executes multiple bin operations atomically and in isolation as a single record command".[22] The map increment "increments values by `delta-value` for all items identified by `key`", and the range removal deletes "all `{key: value}` pairs where `map.key >= keyStart` and `map.key < keyStop`".[21] Put together, the logic is roughly this (pseudocode, not Grab's code):

```
operate(key = "card:X|hourly", [
  map_increment(bin = "counts", key = bucket(now), by = 1),
  map_remove_by_key_range(bin = "counts",
                          from = 0, to = now - retention),
  ...  # Grab also writes the counter key
])
```

Two things fall out of this design. The lost-update problem disappears, because the increment happens on the server inside one record-level atomic command, so there's nothing to interleave with. And the map prunes itself on every write, so it never grows without bound. Grab still sets a record-level TTL *≈ time to live: an expiry time after which the database deletes a record automatically*, but only as a safety net for counters that stop receiving writes, because one record now spans many timestamps.[15]

Step 3: reads, batched per granularity

On the old backend, each sub-query was a prepared statement run concurrently and returning paginated row streams, so one request could make many round trips. On Aerospike, Grab groups the sub-queries by granularity and issues one `BatchOperate` per granularity, so a request makes **at most three round trips**.[15]

Each read fetches the record and sums the map entries in the requested window on the client side; InfoQ points out that this replaces the server-side range filtering the wide-column store used to do.[16]

There's a trade here too. The old backend streamed rows; Aerospike's batch either succeeds or fails as a unit, and there's no streaming.[15] For counters that's fine, because each record is small. It would matter more if a map grew towards the 1 MiB record limit.

What changed, in Grab's numbers

~50% lower production **p99** read latency	**3 → 1 TB** on-disk data	**45–50%** lower **cost per node**	**RF 3 → 2** copies of each record	**≤ 3** database round trips per read request	**~50 / 100 GB** primary index RAM used / usable per node

All figures as reported by Grab (blog, 3 Jul 2026); not independently verified.

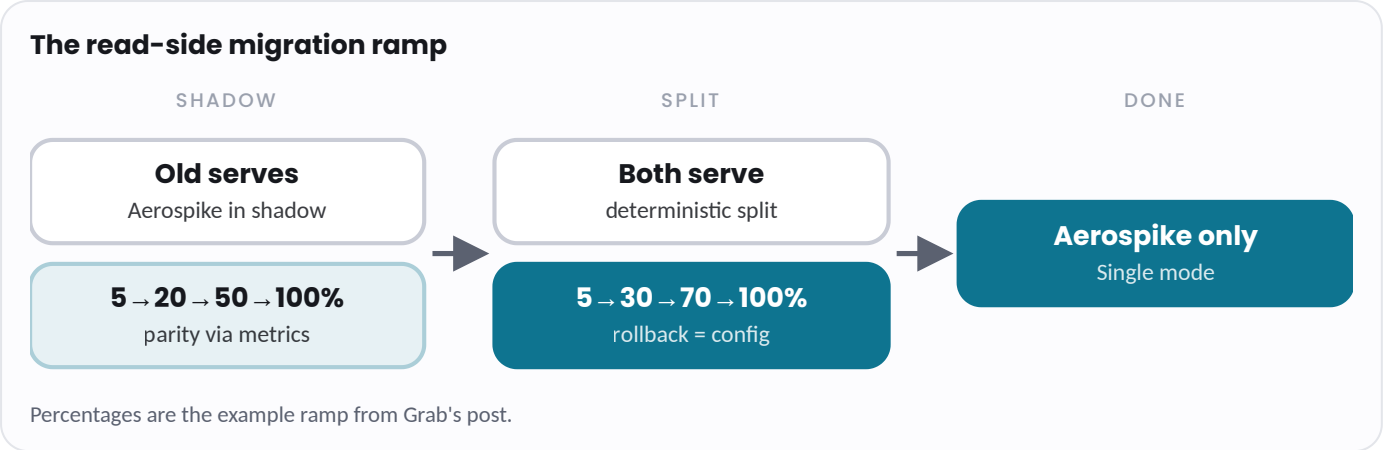
Step 4: migrating a live fraud system without a gap

You can't take the fraud platform offline for a weekend. Grab's requirements were a gradual traffic ramp with rollback through a configuration change, monitoring of both storage paths to verify data before switching, and zero downtime.[15]

The service is written in Rust, and the team prepared it in three stages.[15]

1. **Isolate the driver.** All storage code moved behind a `LegacyStorage` wrapper, so business logic no longer imported the database driver directly. No behaviour changed.
2. **Add a facade with modes.** A storage facade dispatches over Legacy, Aerospike and Mock backends. Grab used Rust enum dispatch *≈ choosing an implementation with a `match` on an enum value, which the compiler can inline, instead of calling through a trait object on the heap* rather than trait objects, to avoid a heap allocation per query. Configuration picks one of three modes: **Single** (one backend serves), **WithShadow** (the primary serves responses while the secondary runs asynchronously for parity checks) and **WithSplit** (a deterministic percentage of live traffic goes to each backend).
3. **Tag the metrics.** Every metric was tagged by backend, role and mode, so existing dashboards kept working and could compare the two paths side by side. Shadow calls were fire-and-forget, so a slow Aerospike couldn't slow down real responses.

Shadow traffic *≈ sending a copy of real requests to a new system without using its answers, so you can compare results and performance under production load* is what made the cutover low-risk. The example ramp in the post goes from shadow reads at 5% → 20% → 50% → 100%, then live split traffic at 5% → 30% → 70% → 100%, then Aerospike only.[15] On the write side, the old backend stayed primary while Aerospike received asynchronous shadow writes under the same deterministic rollout logic.[15]



The experiment that didn't work: index on NVMe

Remember that Enterprise Edition can store the primary index on an NVMe SSD instead of RAM.[19] Grab tried it to raise the memory ceiling. It caused p99 spikes that tracked I/O on **hot keys** rather than overall request rate, and adding nodes didn't help. The team went back to an in-memory index on memory-optimised instances.[15]

That result makes sense once you think about what a **hot key** *≈ a single key that receives a disproportionate share of traffic, such as a very active account* does. Adding nodes spreads *different* keys across more machines, but every request for one hot key still lands on the same partition and, with the index on disk, the same device. With the index in RAM, the hot key's index lookup is a memory access; on NVMe, it competes for the same device queue as everything else. The post's lesson: hot-key I/O can dominate latency when the index is on disk.[15]

Today the primary index uses about 50 GB of roughly 100 GB usable memory per node, which is the headroom the map redesign bought back.[15]

Two client-library surprises

The first Aerospike Rust client Grab used was synchronous, so calls were wrapped with `spawn_blocking` to keep them off the async runtime's worker threads. When the official async client shipped, they removed the bridge and saw p50 and p99 improve.[15]

The second was subtler: the client resolved seed hostnames only once, at start-up. After a staging cluster was replaced with new IP addresses, the service kept trying the old IPs until it was restarted. Grab reported it upstream, a later client release fixed it, and the team now reproduces the scenario in a Docker-based end-to-end test and in staging drills.[15] It's a reminder to test infrastructure replacement, not just node failure, before you depend on a new client library.

WHAT IT MEANS FOR YOU

Design for the per-record cost of your database. In Aerospike, every record costs 64 bytes of RAM per replica.[20] Grab's lesson generalises to any store with a fixed cost per key: when a database charges you per key, shaping the data so you have fewer, fuller keys can matter more than which engine you pick. Aerospike's own docs default to consolidation for exactly this reason, while warning against letting a single record grow too large.[20]

Push read-modify-write to the server. If your counter, rate limiter or quota service reads a value, changes it in application code and writes it back, look for a server-side atomic operation instead (Aerospike map increments, Redis `INCRBY` / `HINCRBY`, Cassandra counters, DynamoDB `UpdateItem` with `ADD`). It's usually both faster and more correct.

Copy the migration pattern even if you never touch Aerospike. A storage facade with Single, Shadow and Split modes, metrics tagged by backend and role, and ramps controlled by configuration is a reusable template for any storage swap.[15]

Know which risk you're accepting when you cut replicas. Going from three copies to two saved a third of storage and index memory, and Grab explicitly accepts the risk of a simultaneous multi-zone failure because the counts can be rebuilt from the event stream.[15] That justification only holds if you really have a durable source to rebuild from.

🌍 In the real world

Daniel Lim, an engineering leader at Grab, summed up the approach as "combining dual read-write paths for traffic shadowing and data parity validation with gradual traffic migration".[16] The other number worth remembering is the one Grab chose *not* to chase: putting the index on NVMe would have raised the memory ceiling, but it made tail latency depend on hot keys, so they bought memory-optimised instances instead.[15]

💥 Watch out for

- **These are Grab's numbers.** The 50% p99, 3 TB → 1 TB and 45–50% cost figures come from Grab's post; InfoQ didn't verify them independently, and the post doesn't name the old database or give its configuration.[15,16]
- **Maps move filtering to the client.** The wide-column store filtered by time range on the server; with maps, the whole record comes back and the client sums the window.[16] That's cheap for small maps and expensive for large ones.
- **Record size limits are real.** `max-record-size` defaults to 1 MiB (ceiling 8 MiB), and writes over the limit fail.[20] Pruning on every write is what keeps Grab's maps small.
- **Index-on-flash is not free memory.** It can tie tail latency to hot-key I/O, as Grab found.[15]

📖 Words from this story

Partition key: the part of the primary key that decides which node stores a row.

Clustering column: a primary-key column that sorts rows within a partition on disk.

Read-modify-write: read a value, change it in the app, write it back; other writers can interleave.

Aerospike: a distributed key-value database with its index in RAM and data on SSD by default.

Record: Aerospike's unit of storage, a key plus named fields.

Bin: a named field inside an Aerospike record.

Collection data type: a list or map inside one bin that the server can modify in place.

Hybrid Memory Architecture: Aerospike's default layout, index in memory and data on SSD.

Red-black tree: a self-balancing binary search tree.

Replication factor: how many copies of each record the cluster keeps.

Secondary index: an extra index on a non-key field.

Sorted map: a map kept in key order, so lookups and key ranges are efficient.

TTL (time to live): an expiry time after which a record is deleted automatically.

Enum dispatch: choosing an implementation via a `match` on an enum instead of a heap-allocated trait object.

Shadow traffic: copying real requests to a new system without using its answers, to compare.

Hot key: one key receiving a disproportionate share of traffic.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Percona's Valkey-proxy (The New Stack, 8 Oct):** an open source proxy that lets apps written for a single Redis instance talk to a Valkey Cluster unchanged, now approved to move into the Valkey project, with full source due end of October, an RC in December and GA in early 2027.[23]
- **Kafka 4.4.0 still not announced:** as of 11 Oct the newest post on the Apache Kafka blog is still 4.2.2 (29 Sep).[24]
- **PostgreSQL 19 RC still not out:** the newest postgresql.org news item is the pgjdbc security release (posted 9 Oct); Beta 4 (24 Sep) had promised an RC in "early October".[25]
- **ClickHouse patch wave (6–8 Oct):** new builds across four supported lines, including v26.9.13.15-stable and v26.8.21.10-lts on 8 Oct.[26]
- **Databricks (9 Oct):** a managed Lakeflow Connect connector for Google Workspace audit activity (Beta), and request tags that Unity Gateway service policies can require before a model or MCP tool call proceeds (Beta).[27]
- **Snowflake (8 Oct):** stable egress IP addresses on Azure are generally available, so outbound connections from Snowflake can be allowlisted by fixed address.[28]
- **Apache data-lake projects (week to 7 Oct):** DataFusion Comet 1.1.0 and Iceberg Go 0.7.0 passed their release votes, and Arrow 26.0.0 RC0 collected five binding approvals.[29]

1. **Release apache-iceberg-1.12.0**, apache/iceberg, GitHub, 30 Sep 2026.
2. **Apache Data Lakehouse Weekly: September 29 to October 7, 2026**, Alex Merced, DEV Community, 7 Oct 2026 (digest of the Apache dev lists, with links to each thread).
3. **Apache Iceberg 1.12.0 Release**, Apache Iceberg PMC, 5 Oct 2026.
4. **Apache Iceberg 1.12.0: What's New, Breaking Changes, and Upgrade Guide**, Alex Merced, Dremio, 30 Sep 2026.
5. **Iceberg Table Spec (format/spec.md)**, Apache Iceberg, main branch, viewed 11 Oct 2026: format versioning, row-level deletes, Variant.
6. **Iceberg Table Spec (format/spec.md)**, Apache Iceberg, main branch, viewed 11 Oct 2026: scan planning, manifest fields, Variant bounds.

7. **Deletion Vectors vs Position Deletes vs Equality Deletes: The Iceberg Delete Story in 2026**, Alex Merced (Dremio), 24 Aug 2026.
8. **The Apache Iceberg v3 Table Spec: Celebrating the Open Source Community's Shared Success**, Snowflake, 10 Jun 2025.
9. **What's New in Apache Iceberg 1.10.0 and What Comes Next**, Dremio, 12 Sep 2025.
10. **May 7, 2026: Support for Apache Iceberg version 3 (General availability)**, Snowflake documentation.
11. **Amazon Redshift now supports Apache Iceberg v3 tables**, AWS What's New, 31 Aug 2026.
12. **Announcing Apache Iceberg 1.11.0**, Google Open Source Blog, 27 May 2026.
13. **Variant Type in Apache Parquet for Semi-Structured Data**, Apache Parquet blog, 27 Feb 2026.
14. **Variant Shredding**, Apache Parquet format documentation.
15. **Migrating Counter Service storage: design choices and learnings**, Jia Long Loh, Md Riyadh, Runze Cui and Yuanzhe Liu, Grab Tech Blog, 3 Jul 2026.
16. **Grab Redesigns Counter Service Storage for 50% Lower P99 Latency**, Leela Kumili, InfoQ, 7 Oct 2026.
17. **CREATE TABLE**, Apache Cassandra documentation.
18. **Hybrid storage**, Aerospike documentation.
19. **Primary index**, Aerospike Database 8.0 documentation.
20. **Record sizing**, Aerospike documentation.
21. **Map operations**, Aerospike documentation.
22. **Single-record commands**, Aerospike documentation.
23. **Redis by proxy: Percona targets the last major hurdle to Valkey adoption**, Paul Sawers, The New Stack, 8 Oct 2026.
24. **Apache Kafka Blog**, viewed 11 Oct 2026.
25. **PostgreSQL News Archive**, viewed 11 Oct 2026; **PostgreSQL 19 Beta 4 Released**, 24 Sep 2026.
26. **ClickHouse releases**, GitHub, viewed 11 Oct 2026.
27. **October 2026 release notes**, Databricks documentation, viewed 11 Oct 2026.
28. **Snowflake server release notes and feature updates**, Snowflake documentation, viewed 11 Oct 2026.
29. **Apache Data Lakehouse Weekly: September 29 to October 7, 2026**, Alex Merced, DEV Community, 7 Oct 2026.